NEBIUS ACADEMY

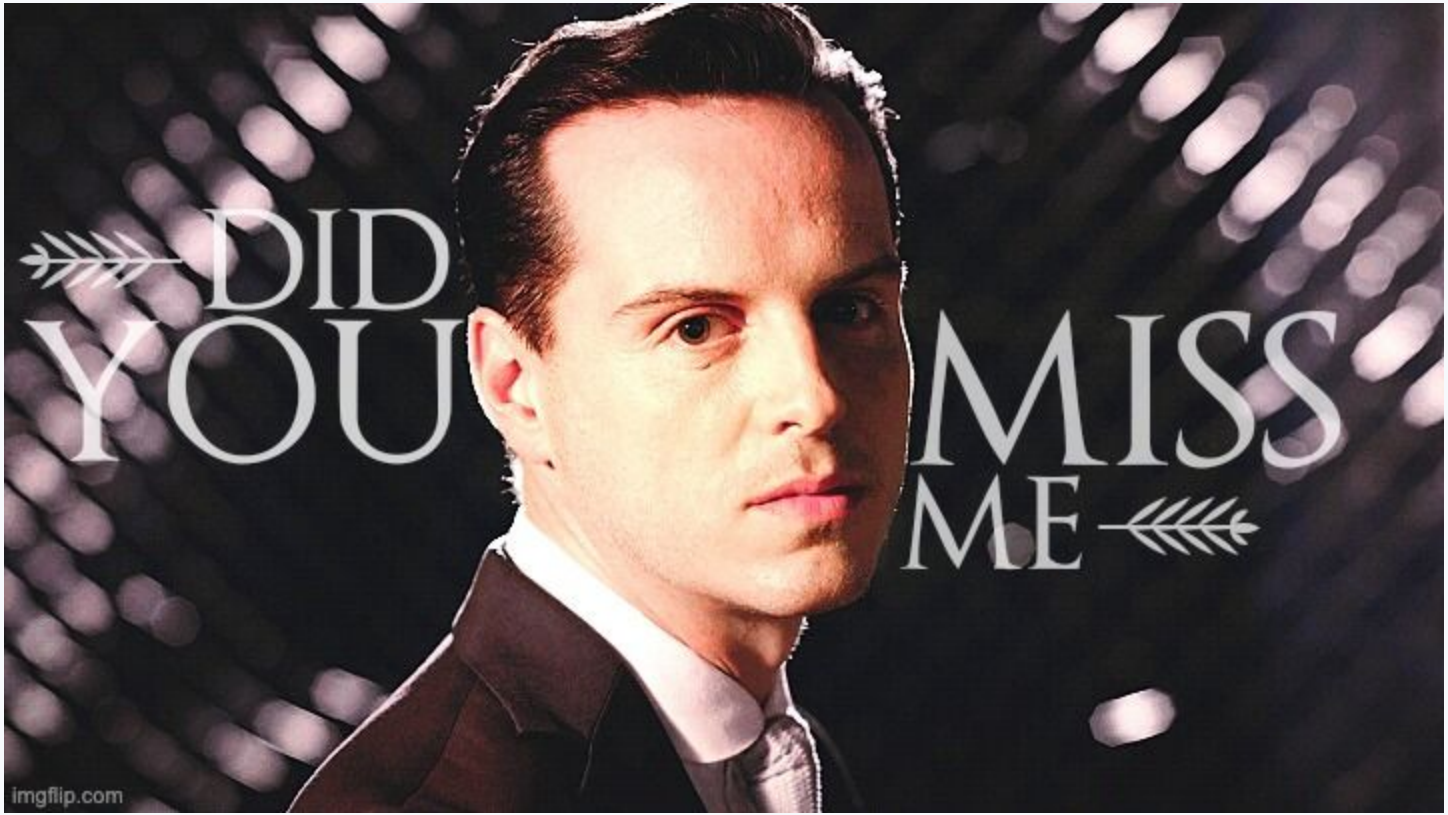
Workflows, Systems & AI Agents

It's not an AI agent unless it comes from the agentic region of Palo Alto. Otherwise, it's just an AI-enabled workflow

Yuval Belfer



May 2026


DID
YOU
MISS
ME
imgflip.com

Module 1 - Status Check

01	02	03	04	05
Intro to AI & LLMs	Evaluation & Benchmarks	RAG	Workflows & Agents	Advanced Agents (Tom)

Last lecture

We taught the LLM to use external knowledge - RAG, evals on retrieval, the RAG triad.

Today

We teach it to act - tools, orchestrated workflows, agent loops, and how to evaluate the result.

Today's Agenda

01 **Workflows & AI Systems**

Tool use · MCP teaser · the AI workflow patterns

02 **Agents: ReAct & friends**

Agent loops · Plan-and-Execute · ReWOO · Reflexion · CodeAct

03 **Evaluating Agents**

Trajectories · benchmarks · why leaderboards leak

04 **Summary & What's Next**

Goodbyes · lecture 5 teaser

NEBIUS ACADEMY

AI Systems: Origin Story

Why LLMs are not enough?

Why LLMs alone are not enough

What LLMs do well

- Language fluency
- General knowledge
- Local reasoning
- Quick prototyping

What they struggle with

- Multi-step tasks
- External knowledge or memory
- Precise formats (JSON, SQL)
- Verification & safety

ChatGPT ≠ GPT-5.5 | Claude ≠ Claude Opus 4.7

Why LLMs alone are not enough

What LLMs do well

- Language fluency
- General knowledge
- Local reasoning
- Quick prototyping

What they struggle with

- Multi-step tasks
- External knowledge or memory
- Precise formats (JSON, SQL)
- Verification & safety

ChatGPT ≠ GPT-5.5 | Claude ≠ Claude Opus 4.7
→ The product does the heavy lifting: memory, tools, routing, safety, UI. The model is just one part.

Solution: give the LLM tools. Let it plan, call, and integrate.

Function Calling, a.k.a. Tool Use

The LLM doesn't have to do everything itself.

It can hand off to deterministic code - a database query, a search API, a calculator, a weather service - and bring the result back into its reasoning.

Tool

A function the LLM can call: name, description, input schema.

Decision

Given a task, the LLM decides whether to call a tool - and which one.

Orchestration

Your code runs the tool, feeds the result back, repeats until done.

LLM + tools = capability

The user asks: "What's the exchange rate USD → EUR?"

Without tools

- Returns outdated information
- Or refuses to answer
- Or hallucinates a confident-sounding number

(Pre-trained models freeze in time. The real exchange rate moved.)

LLM + tools = capability

The user asks: "What's the exchange rate USD → EUR?"

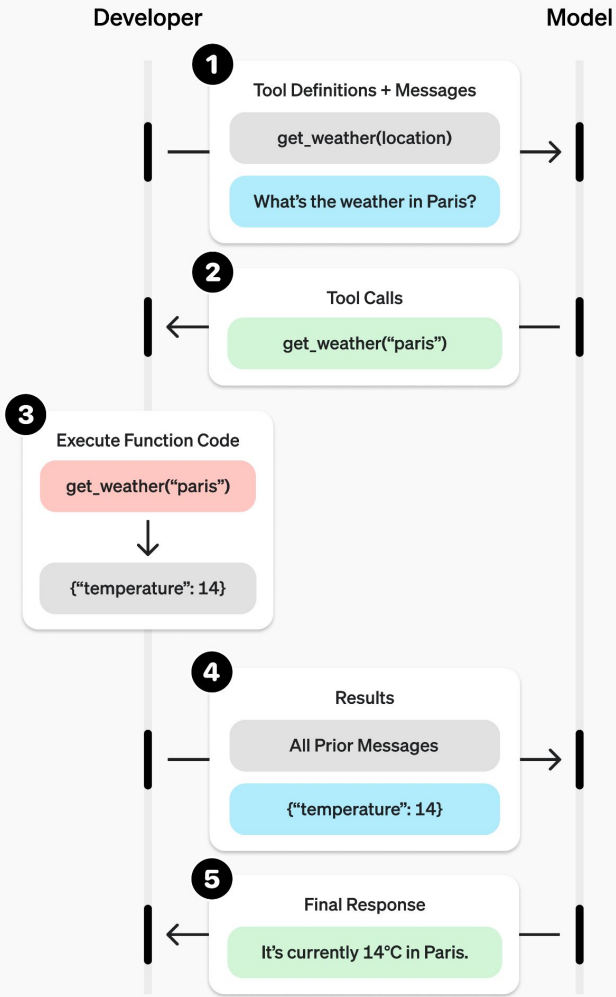
Without tools

- Outdated info
- Refuses to answer
- Hallucinates a rate

With tool use

- LLM decides: I need a currency tool
- Calls get_exchange_rate("USD", "EUR")
- API returns 0.927
- LLM answers with the live rate

Same model. Same prompt. Just access to one tool.



https://developers.openai.com/api/docs/guides/function-calling

Is it that easy?

You still own the orchestration loop.

1	**Provide tools to the model**	It can only use what you give it. No surprise APIs.
2	**Prompt the LLM with the task**	Tools are described in the system prompt automatically.
3	**Receive a tool_use response**	The LLM returns name + arguments — it does not execute.
4	**You execute the tool**	Your code runs it. Handle errors. Validate output.
5	**Feed the result back to the LLM**	The LLM continues reasoning with the new information.
6	**Loop until the LLM stops calling**	Or until you hit your max-iterations safety net.

MCP (or: tool calling 2.0)

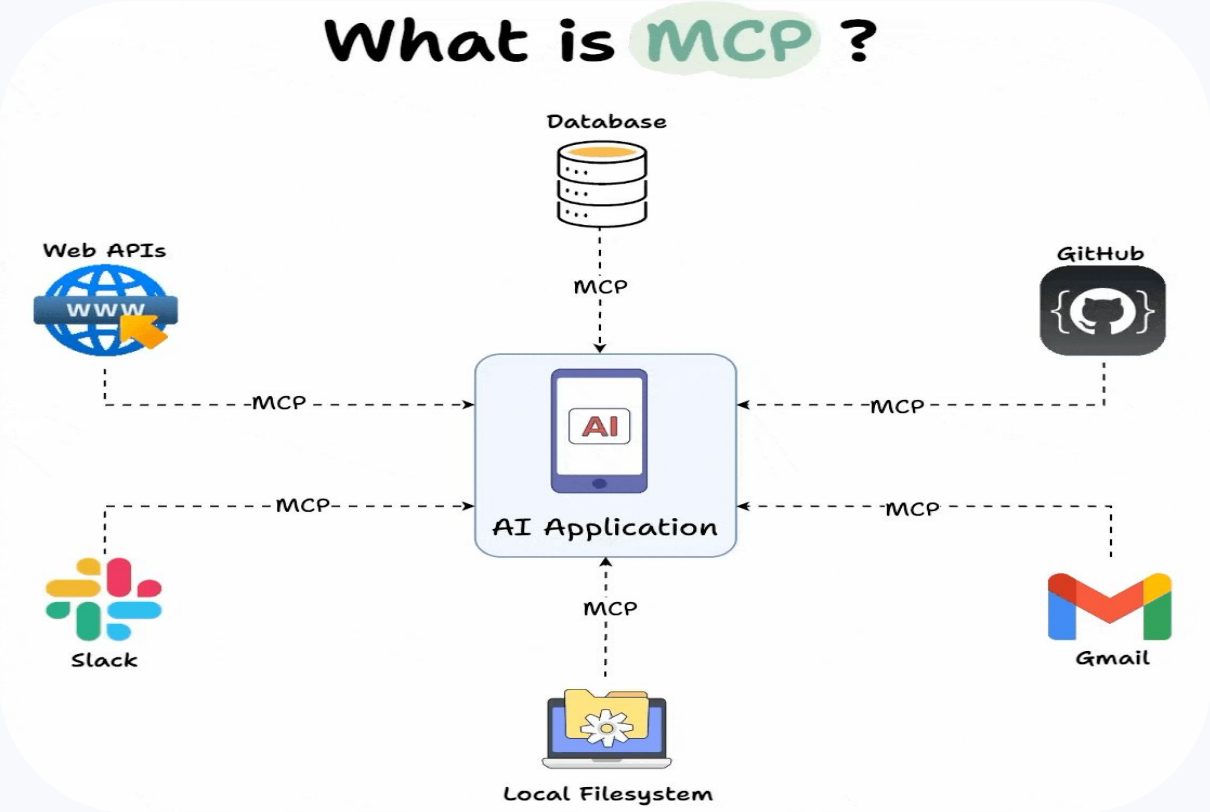
Before MCP

- Every tool integration was bespoke
- Custom code, custom schemas, custom parsing
- n+1 integration problem

After MCP

- One standard protocol
- Tools expose themselves as MCP servers
- Any MCP-compatible model connects

MCP (or: tool calling 2.0)



NEBIUS ACADEMY

Workflows & AI Systems

LLMs are not enough - but do you really need an agent?

What is a workflow?

Workflow = reliable structure

"A predefined sequence of LLM calls, tool calls, and validations - built for reliability and reuse."

https://www.anthropic.com/engineering/building-effective-agents

What is a workflow?

Workflow = reliable structure

"A predefined sequence of LLM calls, tool calls, and validations - built for reliability and reuse."

The key word is predefined.

You — the developer — wrote the steps. The LLM decides what goes into each box, but it doesn't decide what boxes exist.

https://www.anthropic.com/engineering/building-effective-agents

Workflows shine when...

You know the steps
The task decomposes the same way every time.

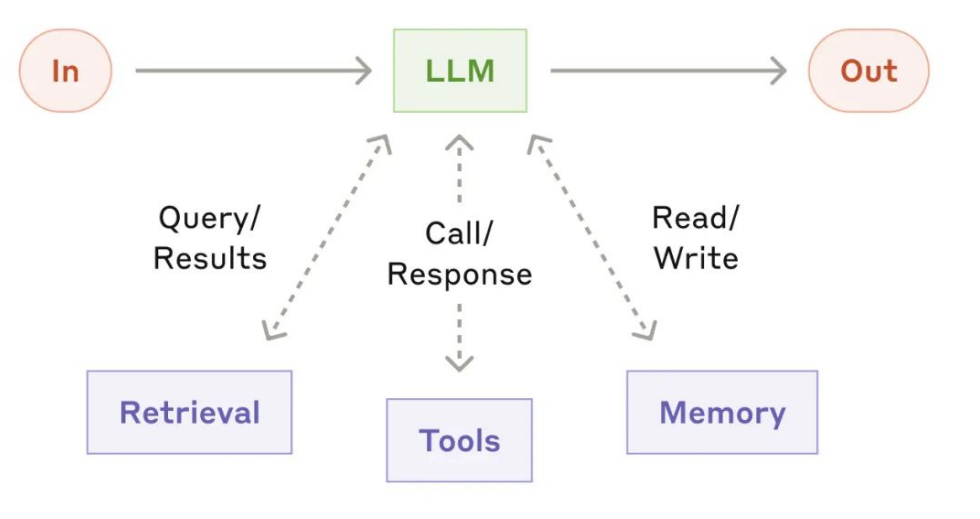
You can define correctness
There's a clear pass/fail or a measurable target.

You care about latency & cost
Predictable cost beats clever-but-variable.

You need reproducibility
Audits, compliance, retrying yesterday's failure.

https://www.anthropic.com/engineering/building-effective-agents

Most basic: the augmented LLM

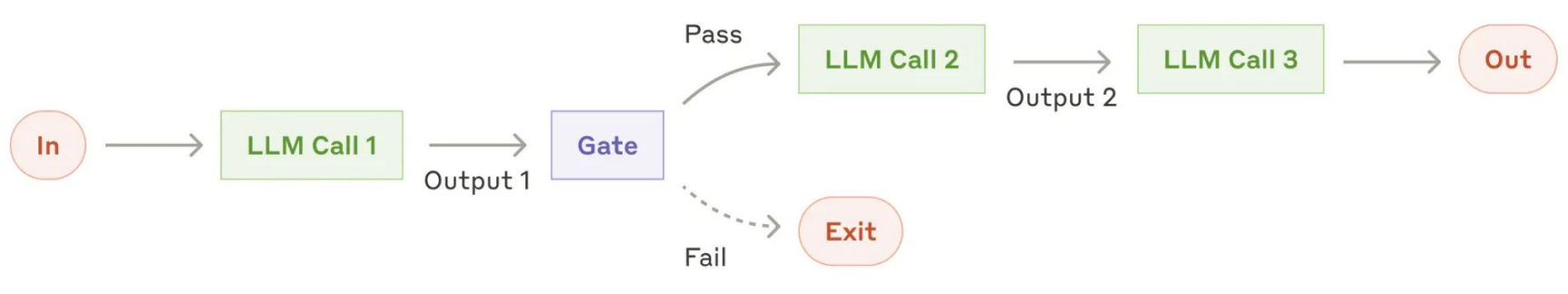


The single LLM, made useful by retrieval, tools, and memory. Every pattern below is built from these.

Example: A chatbot that can search your docs, call your APIs, and remember the user's name across turns.

https://www.anthropic.com/engineering/building-effective-agents

Prompt chaining (a.k.a. static chaining)



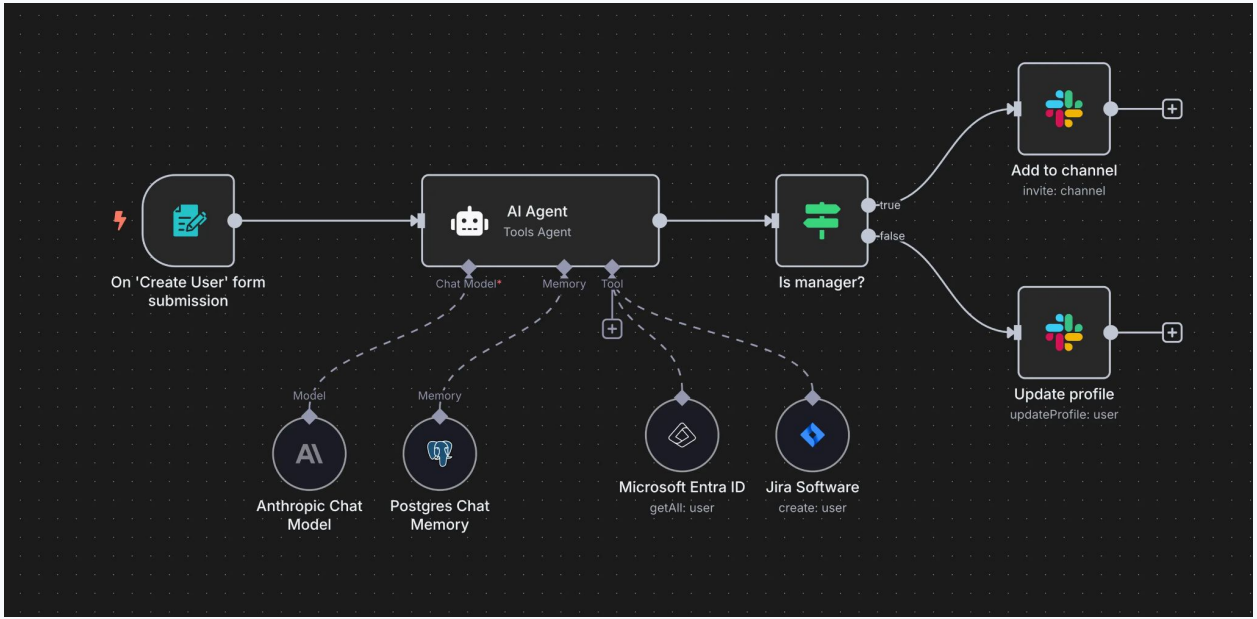
Decompose a task into a fixed sequence. Each step's output feeds the next. Optional gate validates.

Example: Translate marketing copy → tone-check against brand guidelines → format for email.

https://www.anthropic.com/engineering/building-effective-agents

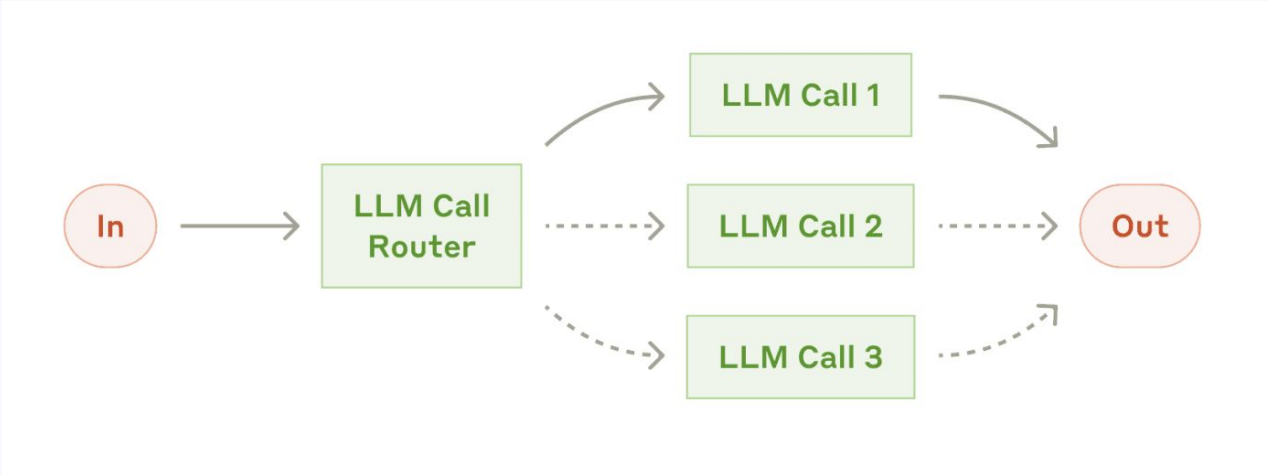
You don't have to code

n8n: a no-code workflow builder



Same idea: nodes = steps, edges = data flow. The LLM is just one type of node among many.

Routing

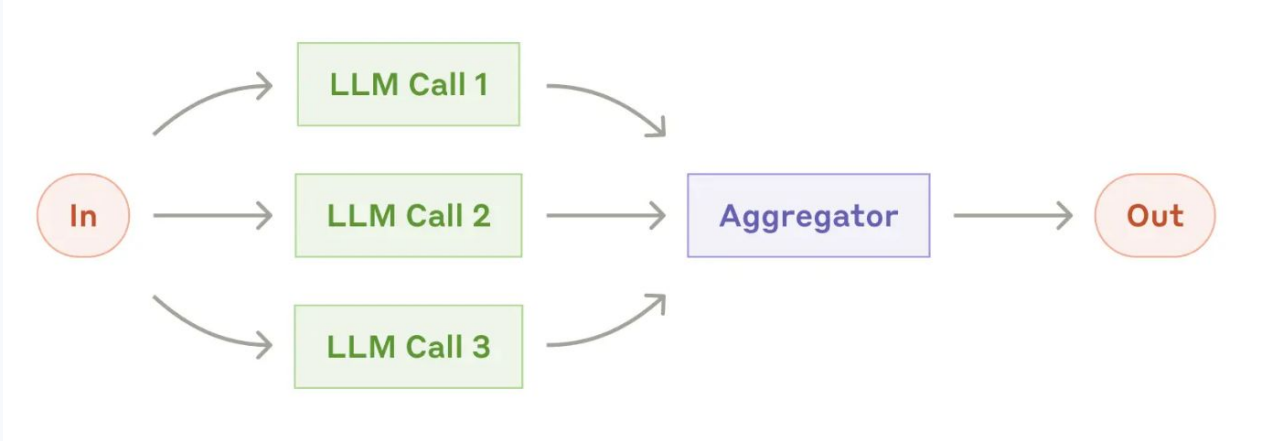


Classify the input first, then dispatch to the specialist (or specialist model) best suited to handle it.

Example: Customer support: simple FAQs → small fast model (Haiku); nuanced complaints → capable model (Opus).

https://www.anthropic.com/engineering/building-effective-agents

Parallelization

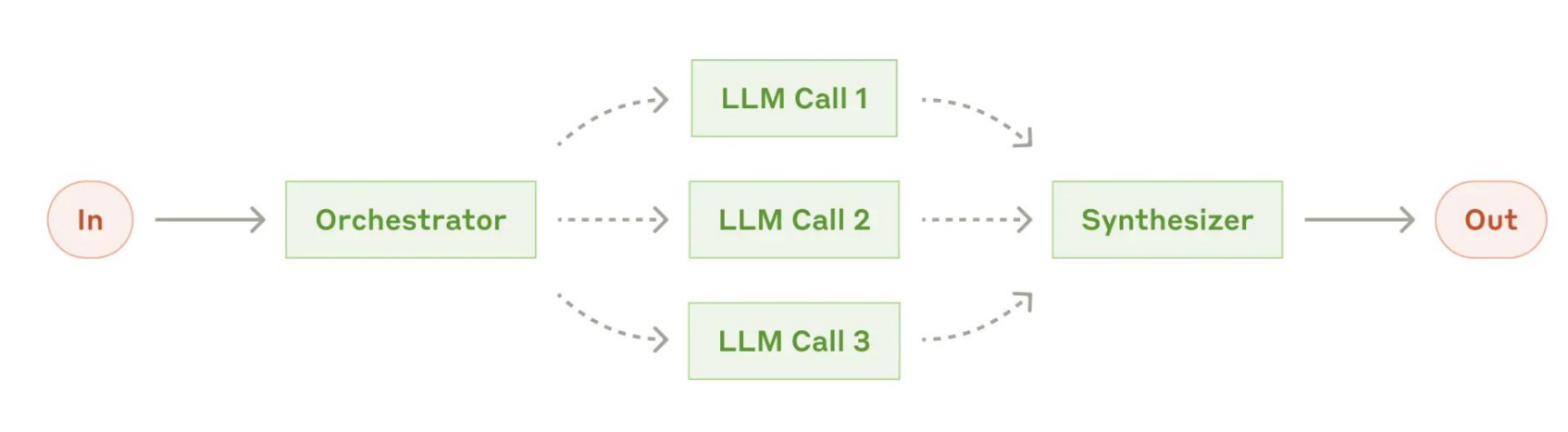


Sectioning: split into independent subtasks. Voting: run the same task N times and aggregate. Both cut latency.

Example: Sectioning: summarize 10 articles concurrently. Voting: 5 model votes for a content-safety label.

https://www.anthropic.com/engineering/building-effective-agents

Orchestrator–workers

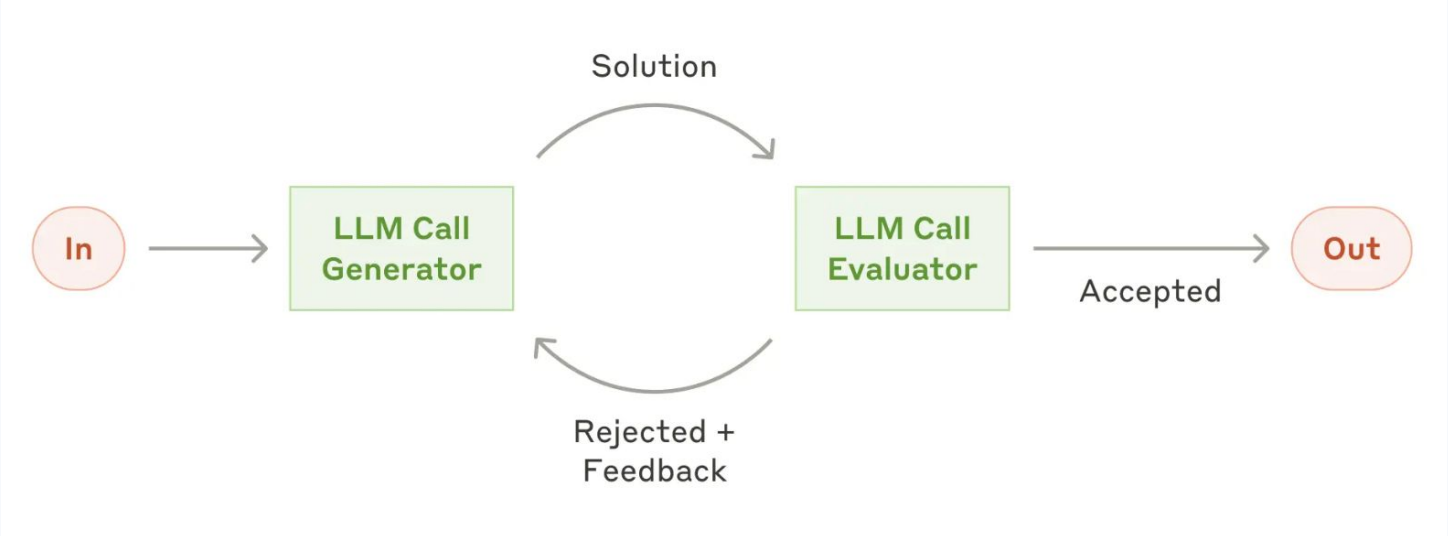


An orchestrator LLM dynamically decomposes a task, dispatches workers, and synthesizes their results.

Example: Research assistant: orchestrator splits a query into 4 sub-queries → workers search → synthesizer drafts the answer.

https://www.anthropic.com/engineering/building-effective-agents

Evaluator–optimizer



Generator drafts; evaluator critiques against criteria; loop until accepted. Trades cost for higher quality.

Example: Code generation: write → run tests → if failing, feed errors back as critique → regenerate.

https://www.anthropic.com/engineering/building-effective-agents

Beyond the original 5

The Anthropic 5 are the foundation. Production systems in 2026 need three more patterns Anthropic's original post didn't formalize.

6 **Human-in-the-loop**

Pause before sensitive actions. Gate, escalate, resume.

7 **Guardrails / validators**

Wrap every LLM call: validate input, validate output.

8 **Reflection / self-critique**

Single LLM plays generator and judge. Cheaper than evaluator-optimizer.

Human-in-the-loop

When stakes are high, pause for a human gate

Pre-action approval

Pause before the action. Surface the proposed call and its arguments. Resume on approve.

Post-action review

Action runs in shadow / sandbox first. Human reviews the result before it ships.

Confidence-based escalation

Only low-confidence outputs route to a human. The rest auto-resolve.

Human-in-the-loop

When stakes are high, pause for a human gate

Pre-action approval

Pause before the action. Surface the proposed call and its arguments. Resume on approve.

Post-action review

Action runs in shadow / sandbox first. Human reviews the result before it ships.

Confidence-based escalation

Only low-confidence outputs route to a human. The rest auto-resolve.

Use when an action is irreversible, regulated, or expensive

Payments · deletions · external messages · compliance-sensitive responses. Implementation: durable state (e.g. LangGraph interrupt + checkpoint) so workflows can pause for hours and resume cleanly.

Guardrails / validators

Wrap every LLM call: validate going in, validate coming out



Input guards check

- PII / secret leakage
- Prompt injection patterns
- Off-topic / out-of-scope

Guardrails / validators

Wrap every LLM call: validate going in, validate coming out

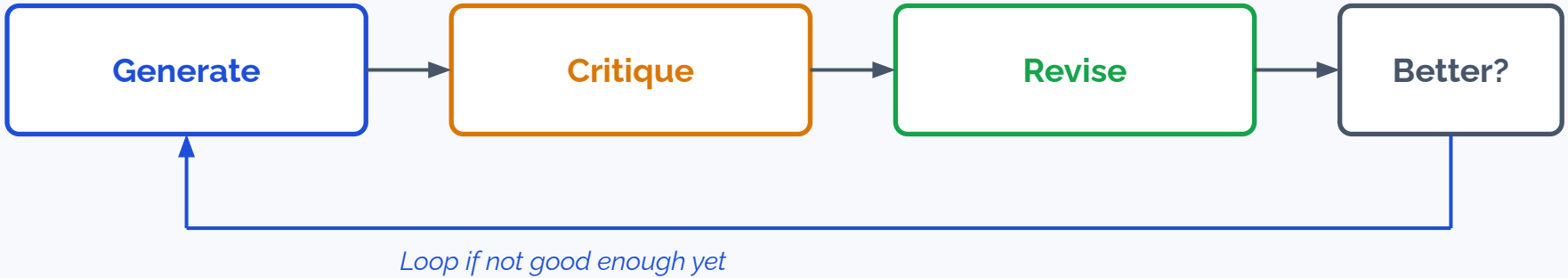


Output guards check

- Schema / JSON validity
- Toxicity / brand safety
- Hallucination signals

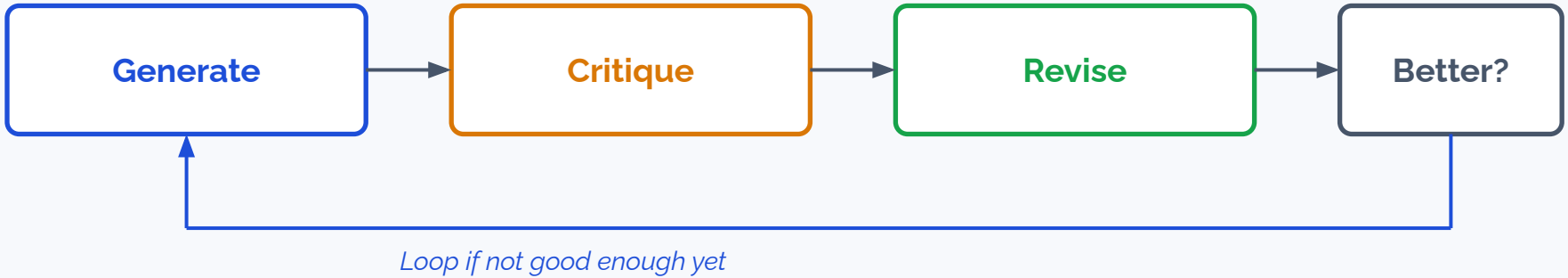
Reflection / self-critique

Same model. Two roles. One loop.



Reflection / self-critique

Same model. Two roles. One loop.

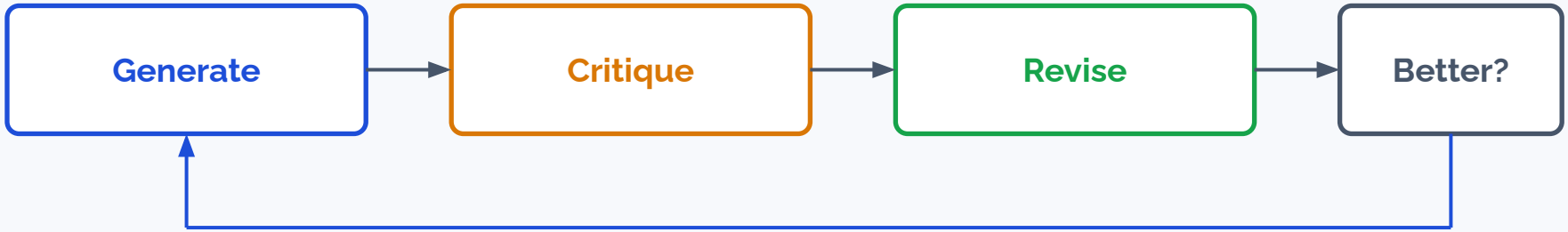


vs. evaluator–optimizer

- Same LLM plays both roles
- Cheaper, simpler scaffolding
- Risk: model can't see its own blind spots

Reflection / self-critique

Same model. Two roles. One loop.



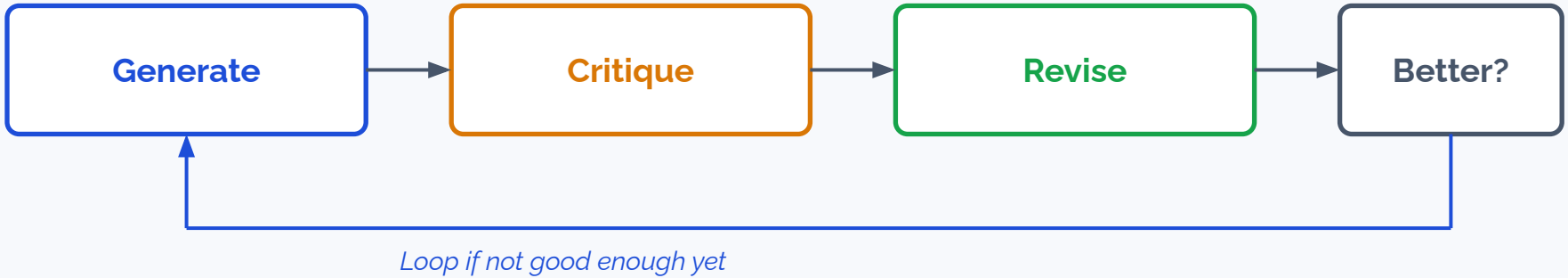
vs. evaluator–optimizer

- Same LLM plays both roles
- Cheaper, simpler scaffolding
- Risk: model can't see its own blind spots

Prefer evaluator-optimizer when you have an external check (tests, judges). Use reflection when you don't.

Reflection / self-critique

Same model. Two roles. One loop.



Where it shines

- Drafting: emails, docs, plans
- Code review on own first draft
- Stepping-stone to Reflexion (later)

Picking a pattern

Pattern	Use when…	Avoid when…
Prompt chaining	Steps are known and ordered	Steps depend on runtime decisions
Routing	Inputs split into clear classes	Class boundaries are fuzzy
Parallelization	Subtasks are independent	Subtasks share state mid-flight
Orchestrator–workers	Decomposition is dynamic, runtime	Static plan is enough
Evaluator–optimizer	External judge is available	No way to score the output
Human-in-the-loop	Action is irreversible / regulated	High volume, low stakes
Guardrails	Untrusted input or output schema	Internal-only, low-risk pipelines
Reflection	No external judge, cost-sensitive	Stakes high enough to warrant evaluator

"Add complexity only when it demonstrably improves outcomes."

? Quiz Time! ?

Test Your Knowledge & Have Fun!



Get Ready to Participate!

Q1: An LLM needs to: translate marketing copy into French, then check tone, then format for email. Which pattern fits best?

A) Parallelization — run all three simultaneously

B) Routing — classify the input and send it to the right handler

C) Prompt chaining — fixed sequence where each step feeds the next

D) Orchestrator-workers — dynamically assign subtasks

Q1: An LLM needs to: translate marketing copy into French, then check tone, then format for email. Which pattern fits best?

A) Parallelization — run all three simultaneously

B) Routing — classify the input and send it to the right handler

C) Prompt chaining — fixed sequence where each step feeds the next

D) Orchestrator-workers — dynamically assign subtasks

Q2: A support bot: 80% are simple FAQs, 20% complex complaints. What's the most efficient architecture?

A) Use the most powerful model for all queries to guarantee quality

B) Use a small model for everything to minimize cost

C) Routing — classify queries first, send easy ones to a small model, hard ones to a capable model

D) Parallelization — run both models on every query and pick the better answer

Q2: A support bot: 80% are simple FAQs, 20% complex complaints. What's the most efficient architecture?

A) Use the most powerful model for all queries to guarantee quality

B) Use a small model for everything to minimize cost

C) Routing — classify queries first, send easy ones to a small model, hard ones to a capable model

D) Parallelization — run both models on every query and pick the better answer

Q3: A bank's agent moves money between accounts. The bank requires explicit user approval for transfers over $1,000. Which pattern fits?

A) Routing — classify by transfer size

B) Evaluator–optimizer — generate then review

C) Human-in-the-loop with confidence-based escalation

D) Reflection — let the agent self-check

Q3: A bank's agent moves money between accounts. The bank requires explicit user approval for transfers over $1,000. Which pattern fits?

A) Routing — classify by transfer size

B) Evaluator–optimizer — generate then review

C) Human-in-the-loop with confidence-based escalation

D) Reflection — let the agent self-check

Agents: ReAct & friends

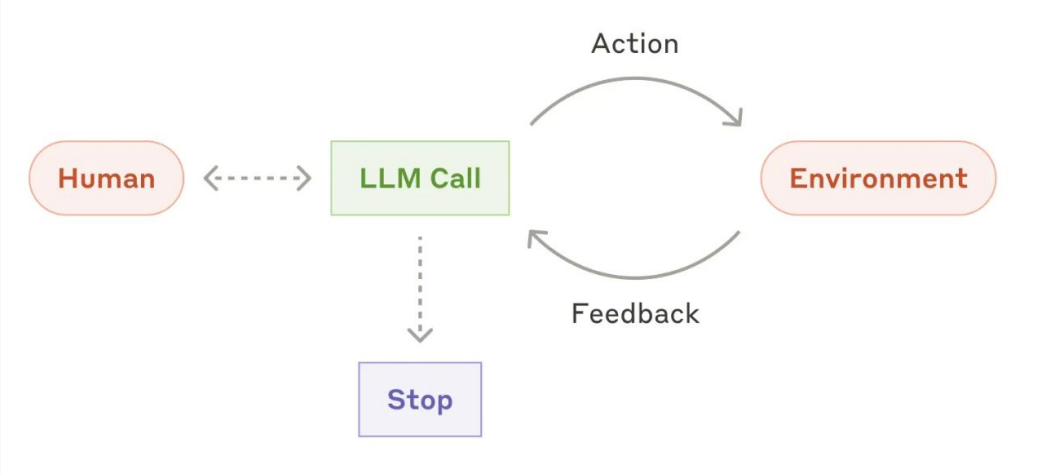
Your favorite's workflow favorite workflow

What is an agent?

Agent = autonomy + memory + tools

"A loop that lets the model plan, act, observe, and decide what to do next — without a human in every loop."

What is an agent?



What is an agent?

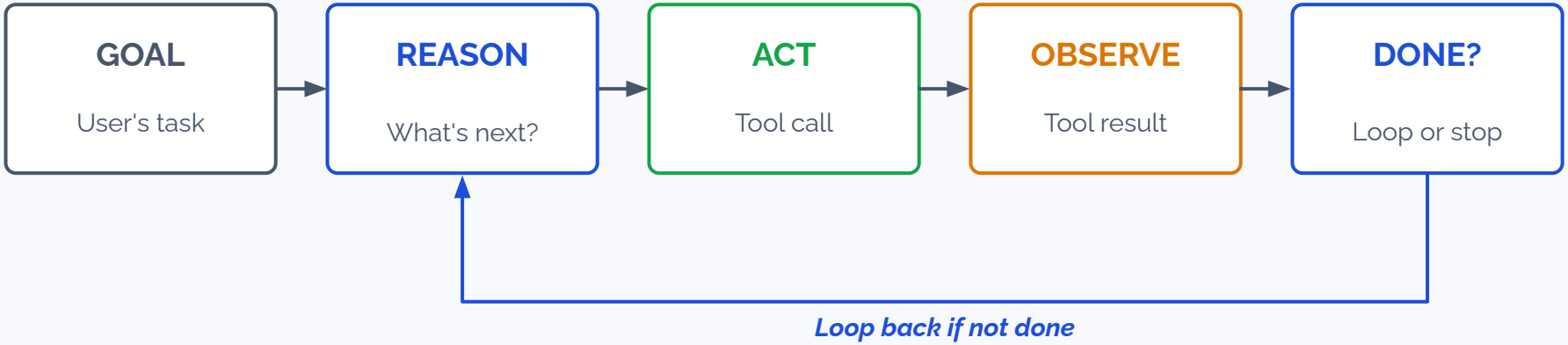
Agent = autonomy + memory + tools

"A loop that lets the model plan, act, observe, and decide what to do next — without a human in every loop."

	Workflow	Agent
Path	Predefined	Decided at runtime
Flexibility	Low — structured	High — open-ended
Cost / latency	Predictable	Variable — can spiral
Reliability	High	Lower — harder to test
Best for	Known tasks, SLAs	Open-ended, research, coding

The agent loop

Every agent variant rearranges the same five steps



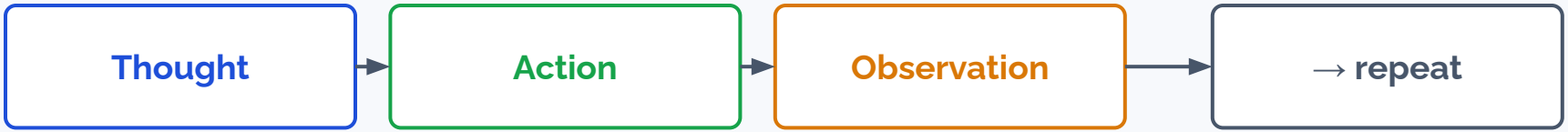
Plan-and-Execute, ReWOO, Reflexion, CodeAct - all variants of where you put the reasoning and how many times you loop.

ReAct: Reasoning + Acting

Yao et al., 2023 — the algorithm that made tool-using agents practical

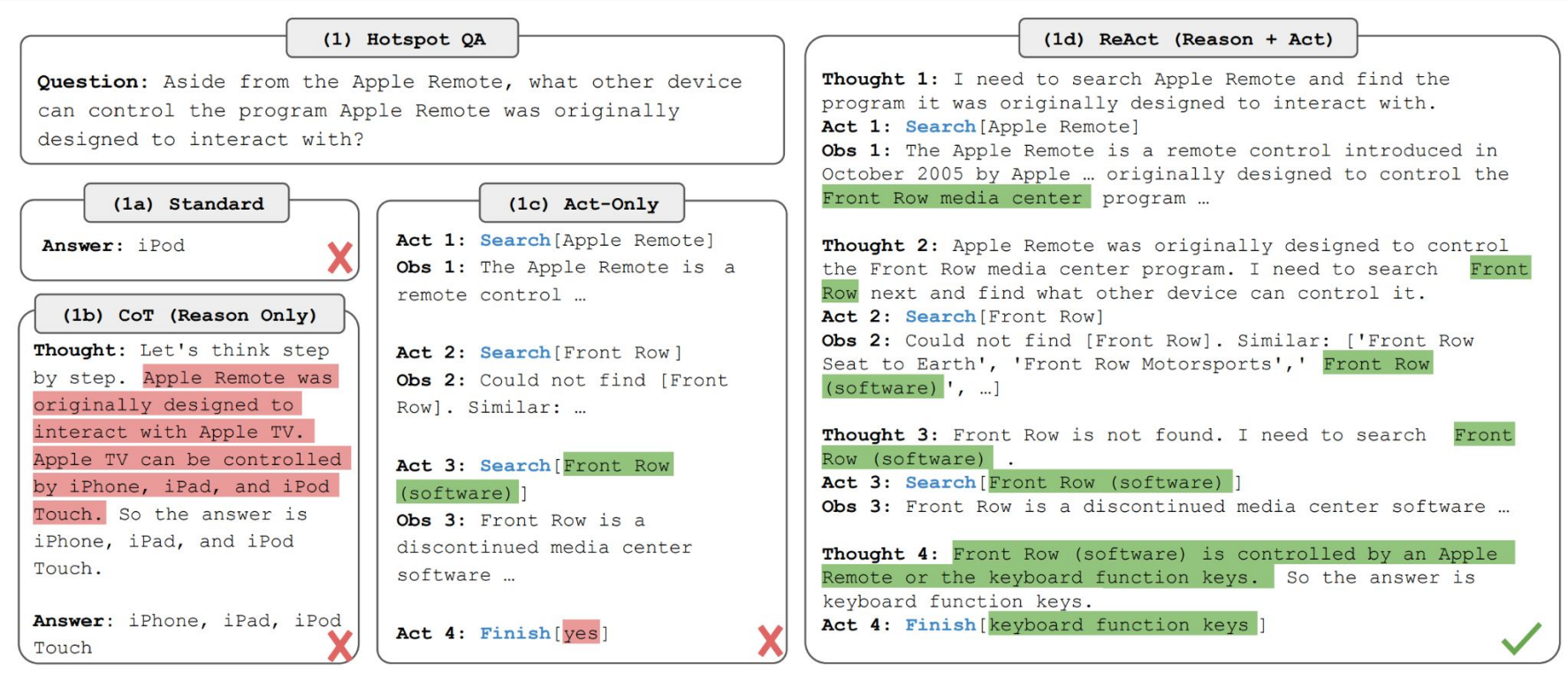
Interleave thinking and doing.

Before each action, the model writes a Thought explaining why. After each action, it Observes the tool result. The trace is its scratchpad.



If someone says 'agent' without further qualification — they almost always mean a ReAct loop.

ReAct: Reasoning + Acting



Yao et. al, ReAct: Synergizing Reasoning and Acting in Language Models, 2023

ReAct: Reasoning + Acting

(2) AlfWorld

You are in the middle of a room. Looking quickly around you, you see a cabinet 6, a cabinet 1, a coffee machine 1, a countertop 3, a stove burner 1, and a toaster 1.
Your task is to: Put some pepper shaker on a drawer.

(2a) Act-Only

Act 1: Go to drawer 1
Obs 1: The drawer 1 is closed.

Act 2: Open drawer 1
Obs 2: You open the drawer 1. The drawer 1 is open. In it, you see a dishsponge 2, and a spoon 1.

Act 3: Go to sinkbasin 1
Obs 3: On the sinkbasin 1, you see a dishsponge 3, a spatula 1, and a spoon 2.

Act 4: Take peppershaker 1 from sinkbasin 1
Obs 4: Nothing happens.

Act 5: Take peppershaker 1 from sinkbasin 1
Obs 5: Nothing happens.



(2b) ReAct (Reason + Act)

Act 1: Think[First I need to find a pepper shaker...more likely to appear in cabinets (1-6), countertops (1-3), ...]
Act 2: Go to cabinet 1
Obs 2: On the cabinet 1, you see a vase 2.

(Here the agent go to cabinet 1, then cabinet 2, 3, then countertop 1 and 2)

Act 6: Go to countertop 3
Obs 6: On the countertop 3, you see a apple 1, a bread 1, ... a pepper shaker 1, and a vase 3.

Act 7: Take pepper shaker 1 from countertop 3
Obs 7: You pick up the pepper shaker 1 from the countertop 3.

Act 8: Think[Now I find a pepper shaker 1. Next, I need to put it in/on drawer 1.]
Act 9: Go to drawer 1
Obs 9: Drawer 1 is closed.

Act 10: Open drawer 1
Obs 10: You open Drawer 1 …

Act 11: Put pepper shaker 1 in/on drawer 1
Obs 11: You put pepper shaker 1 in/on the drawer 1.



Yao et. al, ReAct: Synergizing Reasoning and Acting in Language Models, 2023

ReAct in code

The prompt template the LLM keeps appending to

```
You have access to the following tools:
  - search(query)   : web search
  - calculate(expr) : math

To solve a task, alternate Thought / Action / Observation:

Thought:      <reasoning about what to do next>
Action:       <tool_name>(<args>)
Observation:  <tool result, filled in by the runtime>
... (repeat as needed) ...
Thought:      I have enough information.
Final Answer: <answer>

Begin!
Question: {user_question}
Thought:
```

LangGraph: create_react_agent(model, tools) · smolagents: ToolCallingAgent · raw API: it's just a loop you write yourself.

Live demo



Why ReAct works — and when it breaks

Strengths

- Debuggable — every step is logged
- Grounded — each action is a real tool call
- Incremental — can stop early when confident
- Universal — works with almost any tool set
- Runs out of the box in every framework

Weaknesses

- Token-expensive on long chains
- Can loop forever — needs max-iterations
- Fragile if a tool returns garbage
- Sequential by design — slow wall-clock
- Quality depends heavily on tool descriptions

Why ReAct works — and when it breaks

Strengt

- Runs o
  framework

ool

Most production agents running today are ReAct loops

When (NOT) to use agents

The most common mistake: reaching for an agent when a workflow would do

✓ Good for agents

- Open-ended queries with unpredictable steps
- Tool-based exploration (browsing, coding)
- Research assistants, multi-source analysis
- Tasks where you can't pre-define the workflow

✗ Don't over-agent

- Simple tasks with known answers
- SLAs requiring speed/cost guarantees
- When you can just call a function
- When a workflow handles it reliably

When (NOT) to use agents

The most common mistake: reaching for an agent when a workflow would do

✓ Good for agents

- Open-ended queries with unpredictable steps
- Tool-based exploration (browsing, coding)
- Research assistants, multi-source analysis
- Tasks where you can't pre-define the workflow

✗ Don't over-agent

- Simple tasks with known answers
- SLAs requiring speed/cost guarantees
- When you can just call a function
- When a workflow handles it reliably

The best agentic systems are the simplest ones.

Simplify the workflow. Use deterministic logic where possible. Add autonomy only where you truly need it.

Beyond ReAct

Same loop, different trade-offs

Plan-and-Execute — Plan upfront with a strong model. Execute with a cheap one. Replan on failure.

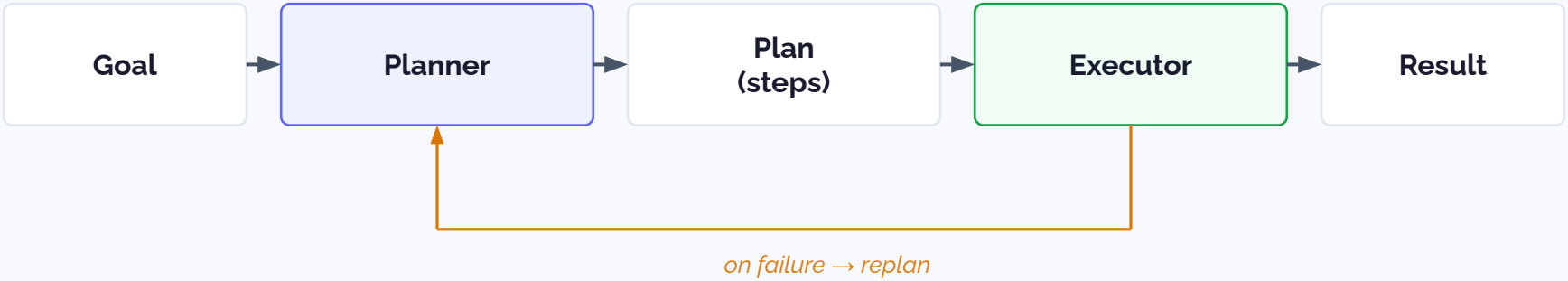
ReWOO — Plan once with placeholders. Run all tools in parallel. Synthesize at the end.

Reflexion — ReAct + a self-critique step. Store critiques in memory across attempts.

CodeAct — Action is executable code, not a fixed tool call. The model writes & runs Python.

Plan-and-Execute

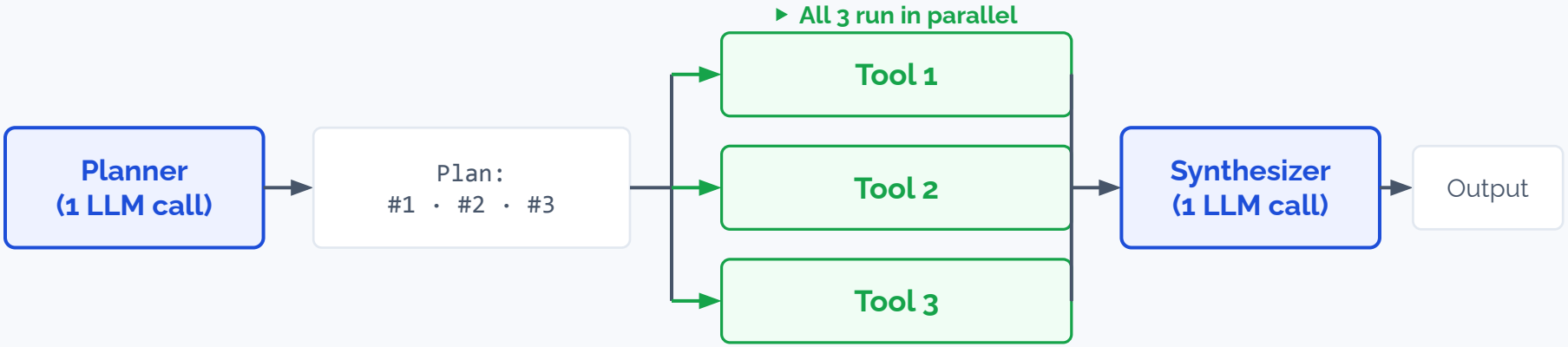
Plan once with a smart model, execute with a cheap one



- ✓ Fewer LLM calls, lower cost
- ✓ Inspectable plan before execution
- ✗ Rigid if plan is wrong; replanning adds complexity

ReWOO: Reasoning Without Observation

Plan once with placeholders. Run tools in parallel. Synthesize.

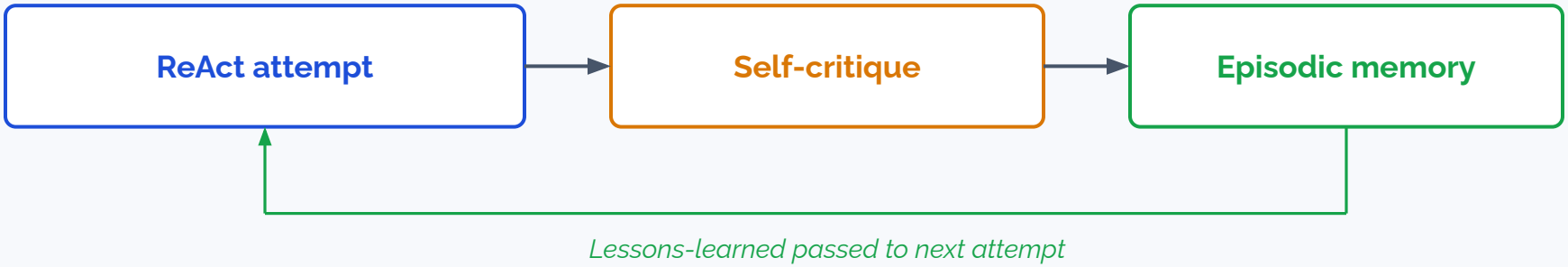


- ✓ Only 2 LLM calls total — ~5x token efficiency vs ReAct
- ✓ Parallel tool calls = lower latency
- ✗ Breaks if a tool returns something unexpected (no mid-flight reasoning to recover)

Xu et. al, ReWOO: Decoupling Reasoning from Observationsfor Efficient Augmented Language Models, 2023

Reflexion

ReAct + memory of past failures



- ✓ Genuinely improves on iterable tasks: code, games, multi-attempt search
- ✗ Highest latency of any single-agent pattern
- ✗ Memory is short-lived — Reflexion is per-task, not lifelong

CodeAct

Action = executable code, not a structured tool call

ReAct action

```
Action: search("Q1 revenue 2025")
Action: search("Q2 revenue 2025")
Action: search("Q3 revenue 2025")
Action: search("Q4 revenue 2025")
Action: calculate(... + ... + ...)
```

CodeAct action

```
quarters = ["Q1","Q2","Q3","Q4"]
revs = [search(f"{q} revenue 2025")
        for q in quarters]
total = sum(parse(r) for r in revs)
print(total)
```

- ✓ Massively more expressive — loops, conditionals, data manipulation
- ✓ This is how Claude Code, smolagents CodeAgent, and modern coding agents work
- ✗ Needs a sandbox. The agent can write anything — including bugs and side effects.

Picking a variant

Pick by the failure mode you can live with

Pattern	LLM calls	Cost	Failure you can live with
ReAct	Many	High	Wasted tokens, occasional looping
Plan-and-Execute	Plan + N exec	Medium	Replanning complexity
ReWOO	2 total	Low	Rigidity if env is unpredictable
Reflexion	Many + critique	Highest	Latency
CodeAct	Variable	Variable	Sandbox risk; debugging code agents

→ *Most production agents are still ReAct or Plan-and-Execute. The exotic ones earn their complexity.*

Building good agents

Hard-won wisdom from teams that ship them

1. **Simplify the workflow first**
   Group two tools into one. Replace agentic decisions with deterministic logic wherever possible.

Building good agents

Hard-won wisdom from teams that ship them

1. **Simplify the workflow first**
   Group two tools into one. Replace agentic decisions with deterministic logic wherever possible.

2. **Improve the information flow**
   Crystal-clear instructions. Log everything inside tools — especially errors.

Building good agents

Hard-won wisdom from teams that ship them

1. **Simplify the workflow first**
   Group two tools into one. Replace agentic decisions with deterministic logic wherever possible.

2. **Improve the information flow**
   Crystal-clear instructions. Log everything inside tools — especially errors.

3. **Tools > prompts**
   Tool design matters more than prompt cleverness. Few thoughtful tools beat many bad ones.

Building good agents

Hard-won wisdom from teams that ship them

1. **Simplify the workflow first**
   Group two tools into one. Replace agentic decisions with deterministic logic wherever possible.

2. **Improve the information flow**
   Crystal-clear instructions. Log everything inside tools — especially errors.

3. **Tools > prompts**
   Tool design matters more than prompt cleverness. Few thoughtful tools beat many bad ones.

4. **The 'dumb engineer' test**
   "How easy would it be for me, with no context, to use this tool correctly and recover from errors?"

Tool description: bad example

If a human can't tell what this does, neither can the LLM

```
def get_data(id: str) -> dict:
    """Get data."""
    ...
```

What kind of data? *What is id? An order? A user? A document?* ***When should the LLM call this — and when should it not?***

Tool description: good example

Be precise about purpose, parameters, and failure modes

```
def get_order(order_id: str) -> dict:
    """Look up an order by its public order ID (e.g. "ORD-9F2A").

    Use this when the user asks about a specific order they've placed:
    status, items, shipping address, total. Do NOT use to list orders
    for a user — use list_user_orders for that.

    Returns:
        On success: {"status": "shipped", "items": [...], "total": 42.50, ...}
        Not found:  {"error": "order_not_found", "order_id": <input>}
    """
```

Quiz Time!

Test Your Knowledge & Have Fun!



Get Ready to Participate!

Q4: You need to draft 50 sales emails from a CRM dump. Each email is independent. Which pattern minimizes cost?

A) ReAct — let the agent figure it out

B) Reflexion — critique each draft

C) Parallelization (workflow) — emails are independent, no agent needed

D) Plan-and-Execute — plan all 50, then run

Q4: You need to draft 50 sales emails from a CRM dump. Each email is independent. Which pattern minimizes cost?

A) ReAct — let the agent figure it out

B) Reflexion — critique each draft

C) Parallelization (workflow) — emails are independent, no agent needed

D) Plan-and-Execute — plan all 50, then run

Q5: Your code-generation agent often produces buggy first attempts. You have an automated test runner. Which pattern wins?

A) ReAct — try, observe failures, retry

B) Reflexion — write code, run tests, store the lesson, retry

C) ReWOO — plan once, run all in parallel

D) Plan-and-Execute — plan the code, then write it

Q5: Your code-generation agent often produces buggy first attempts. You have an automated test runner. Which pattern wins?

A) ReAct — try, observe failures, retry

B) Reflexion — write code, run tests, store the lesson, retry

C) ReWOO — plan once, run all in parallel

D) Plan-and-Execute — plan the code, then write it

Evaluating Agents

Why agent eval is harder than LLM eval — and why most leaderboards leak

The vocabulary of agent evals

Terms you'll see in every serious eval framework

Term	What it means
Task	A single test: defined input + success criteria
Trial	One attempt at a task. Run many — outputs vary
Transcript	The complete record: every tool call, thought, and result
Outcome	The final state of the environment (not what the agent said — what actually happened)

The vocabulary of agent evals

Terms you'll see in every serious eval framework

Term	What it means
Grader	Logic that scores some aspect of performance. One task, multiple graders.
Harness	Infrastructure that runs evals end-to-end: runs tasks, records steps, grades, aggregates
Evaluation suite	A collection of tasks measuring a shared capability area
Scaffold	The agent harness: orchestrates tool calls, processes inputs. Scores measure model AND scaffold together.

Three types of graders

Choose deterministic where you can, LLM where you must, human to calibrate

Code-based (reach for these first)

✓ Fast, cheap, reproducible, objective
✗ Brittle to valid variations; can't handle subjective quality

Model-based (where you need nuance)

✓ Flexible, handles open-ended tasks, scales
✗ Non-deterministic, needs calibration against human judgement, more expensive

Human (the gold standard)

✓ Highest quality, matches real user judgement
✗ Expensive, slow — not for routine use

Don't grade the path, grade the outcome. Agents find valid approaches you didn't anticipate - grading specific tool call sequences punishes creativity.

Agent eval ≠ LLM eval

1

Agents have trajectories

You can be right for the wrong reasons. The path matters, not just the answer.

2

Agents have state

Reads and writes to the world. Eval the final state, not just the final message.

3

Agents have a tool budget

Cost and latency are first-class metrics, not footnotes. Tokens × tool calls × wall-clock.

What to measure

Headline accuracy is one of five — and not always the most important one

Task success	Trajectory quality	Cost	Latency	Reliability
Final state correct?	Right tools, right order?	Tokens, $, tool calls	Wall-clock, especially p95	Consistency across N runs

→ A single eval run is meaningless. The 2026 emphasis is N-run consistency (HAL Reliability, ICLR 2026).

Benchmark families

Each measures something different; never collapse them into one ranking

Coding

SWE-bench (Verified, Pro), Terminal-Bench

Tool use / function calling

BFCL, τ-bench, τ^2-bench (Sierra)

General assistant / web

GAIA, Gaia2, WebArena, BrowseComp, BrowseComp-Plus

Computer use

OSWorld, AndroidWorld, Windows Agent Arena

Deep research

DeepResearch Bench (Du et al.), BrowseComp-Plus (Chen et al.)

→ A 70% on WebArena and a 70% on WebVoyager are not comparable. Different graders, different environments. Deep research agents run for minutes — a different evaluation regime entirely.

SWE-bench

The most-cited coding benchmark — with the most-confounded numbers

What it is

- Real GitHub issues from 12 Python repos
- Verified subset: 500 human-validated tasks
- Programmatically graded with the original test suite
- Pro variant: 1,865 problems from 41 professional repos

The catch

- Score depends on the agent harness, not just the model
- Same Claude Opus: 64.9% in one harness, 57.6% in another (~7 pts!)
- Self-reported vs 3rd-party verified scores often differ
- Always read the harness alongside the score

April 2026: top score on SWE-bench Verified is ~87.6% (Claude Opus 4.7).

τ-bench

The benchmark that looks most like real production work

Why τ-bench is different

Stateful	Mocked database persisted as JSON. Reads and writes both count.
Dual-control	A simulated user actively modifies the same environment as the agent.
Policy-bound	Each domain has explicit policies (refunds, cancellations) the agent must follow.
Domain-realistic	Customer service across retail, airline, telecom — not toy problems.

Agents that ace SWE-bench can still fail τ-bench. Coordination + policy compliance are different muscles.

GAIA & Gaia2

General assistant tasks — chained web browse + file parse + multi-doc reasoning

92%
Human baseline

15%
GPT-4 + plugins (2023)

~75%
Top agents today

Gaia2 (Meta, 2025)

Adds asynchrony, time pressure, multi-agent collaboration, and ambiguous instructions. Agents that handle GAIA fail Gaia2 in new ways — coordination and time-aware planning surface failure modes that static benchmarks miss.

Deep Research Bench

100 PhD-level research tasks across 22 fields — not web Q&A, actual analyst-grade synthesis

- 100 tasks crafted by PhD holders and senior practitioners (5+ years experience)
- 50 English, 50 Chinese, spanning 22 domains (Science & Technology and Business & Finance are the largest)
- Tasks drawn from 96,000+ real user queries requiring multi-step web search and synthesis
- Output is a full research report — not a chat reply

Deep Research Bench

100 PhD-level research tasks across 22 fields — not web Q&A, actual analyst-grade synthesis

Two evaluation frameworks, not one:

- RACE (Reference-based Adaptive Criteria-driven Evaluation) — scores report quality across four dimensions: Comprehensiveness, Depth, Instruction-following, Readability. 72% agreement with human experts — significantly above a vanilla prompt baseline (60%).
- FACT (Factual Abundance and Citation Trustworthiness) — scores how many effective citations the agent gathered and how accurate those citations are. This separates "agents that write fluently" from "agents that actually did the research."

Building your own agent eval

EDD callback: same loop as Week 2, multi-dimensional success criterion

1	**Define success**	Final state + cost + latency + safety. Not just answer correctness.
2	**Build the golden set**	Real user tasks if you have them. Synthesize the rest. Cover edge cases on purpose.
3	**Trace everything**	Every tool call, every LLM call, every observation. You can't eval what you can't see.
4	**Score trajectories**	LLM-as-judge over the trace + state-based checks (DB row exists? file written?)
5	**Run N times**	p50, p95, and consistency. A single pass is theatre.

Building your own agent eval

EDD callback: same loop as Week 2 multi

1 **Def**

5 **Run N t**

"0% pass@100 means broken task, not broken agent."

If your agent fails a task on every single run, the first suspect is your task spec or grader - not the model. Always create a reference solution that proves the task is solvable before blaming the model. Then read the transcripts.

? Quiz Time! ?

Test Your Knowledge & Have Fun!



Get Ready to Participate!

Q6: Your team's coding agent scores 75% on SWE-bench Verified. A competitor's scores 78%. Should you switch?

A) Yes — 3 points is a meaningful gap

B) No — 3 points is within noise

C) Depends — what's the harness, the cost per task, and the N-run consistency?

D) Only if the competitor open-sources their trace

Q6: Your team's coding agent scores 75% on SWE-bench Verified. A competitor's scores 78%. Should you switch?

A) Yes — 3 points is a meaningful gap

B) No — 3 points is within noise

C) Depends — what's the harness, the cost per task, and the N-run consistency?

D) Only if the competitor open-sources their trace

Summary

What we did today?

Three takeaways

01

Most 'agent' problems are workflow problems.

Start with the simplest pattern that works. Add autonomy only when the task genuinely demands it — and you'll be surprised how rarely it does.

02

ReAct is the universal skeleton.

Before this lecture, "just use an agent" was the answer. Now you can pick the right loop for the right trade-off: cost, latency, flexibility, or reliability.

03

Eval is harder than it looks.

Benchmark scores are a starting point, not a verdict. The teams that win long-term instrument their own agents: trace every call, run N times, and build golden sets from real failures.

+1 Takeaway

If you don’t need an agent, don’t use an agent.

Don’t overcomplicate your system just for the sake of having an agent.
Simple is better.

Let's be friends

